

CIVILIZATIONS



Autores: Jairo, Izan y Diego

Civilizations	3
1. Introducción del proyecto	3
1.1.Descripción del juego	3
1.2.Objetivos del proyecto	3
1.3.Herramientas utilizadas	4
2. Análisis y diseño	5
2.1.Especificaciones funcionales	5
2.2.Especificaciones no funcionales	5
2.3.Estructura del proyecto	6
2.4.Diseño de mecánicas	7
2.4.1.Recursos	7
2.4.2.Edificios	7
2.4.3.Tecnologías	7
2.4.4.Batalla	8
2.5.Diagrama de clases	8
2.5.1.Diagrama de secuencia simplificado de una batalla	9
3. Arquitectura y desarrollo	10
3.1.Estructura del código fuente	10
3.2.Componentes principales	11
3.3.Diagrama de archivos	11
3.4 Flujo de datos	12
4. Test y depuración	13
4.1.Pruebas realizadas	13
4.2.Errores encontrados y soluciones	14
4.3.Mejoras aplicadas durante el desarrollo	15
5.Recursos gráficos y de audio	15
5.1 Fuentes utilizadas	15
6.Manual de usuario	16
6.1.Cómo ejecutar el juego	16
6.1.1.Versión por consola (sin interfaz gráfica)	16
6.1.2.Interfaz gráfica (JavaFX)	16
6.2 Controles y objetivo del juego	17
6.2.1.Versión consola	17
6.2.2.Interfaz gráfica	17
6.3 Explicación de las opciones del menú	17
6.3.1.Menú de consola	17
6.3.2.Interfaz gráfica (pestañas)	18
7. Reflexión y mejoras	19
7.1 Dificultades encontradas y cómo se superaron	19
7.2 Mejoras futuras	19

Civilizations

Grupo 5 · Curso 2025-2026

1. Introducción del proyecto

1.1.Descripción del juego

Civilizations es un juego de estrategia en consola donde el jugador gestiona una civilización medieval. Debe administrar recursos (comida, madera, hierro y maná), construir edificios que mejoran la generación de recursos, investigar tecnologías de ataque y defensa, y reclutar un ejército con unidades de infantería, defensas fijas y unidades especiales. Periódicamente, un ejército enemigo ataca la civilización y se desencadena una batalla táctica automática. El objetivo es sobrevivir el mayor número de batallas, optimizando la economía y la composición del ejército.

1.2.Objetivos del proyecto

- Implementar un juego completo en Java con mecánicas de gestión de recursos, combate, progresión y persistencia de datos.
- Desarrollar una interfaz gráfica moderna (JavaFX) que permita al jugador interactuar visualmente con el juego.
- Crear una base de datos relacional (Oracle SQL) para almacenar partidas y batallas.
- Construir una aplicación web (Node.js + Express + Handlebars) que muestre información actualizada de la civilización y el historial de batallas.
- Aplicar control de versiones con Git y GitHub, siguiendo una metodología de ramas.

1.3.Herramientas utilizadas

Herramienta	Versión	Uso
Java (JDK)	17+	Lógica principal del juego
JavaFX	23	Interfaz gráfica
XAMPP SQL	3.3.0	Base de datos relacional
Node.js	20.x	Servidor web
Handlebars (hbs)	4.2.0	Motor de plantillas
CSS3	-	Estilos de la web
Git / GitHub	-	Control de versiones
Visual Studio Code	1.90+	Entorno de desarrollo

2. Análisis y diseño

2.1. Especificaciones funcionales

- El jugador puede construir edificios (granja, carpintería, herrería, torre mágica, iglesia) que incrementan la generación de recursos.
- Puede mejorar las tecnologías de ataque y defensa, cuyo coste aumenta con cada nivel.
- Puede reclutar 9 tipos de unidades (Swordsman, Spearman, Crossbow, Cannon, ArrowTower, Catapult, RocketLauncherTower, Magician, Priest).
- Un ejército enemigo se genera automáticamente cada 3 minutos, usando recursos crecientes según el número de batallas ya libradas.
- Las batallas se resuelven siguiendo un algoritmo detallado con selección probabilística de atacantes y defensores, daños, residuos y cambio de turno.
- Al finalizar cada batalla se almacena un informe completo y se actualizan los recursos de la civilización.

2.2. Especificaciones no funcionales

- Rendimiento: Las batallas deben ejecutarse en tiempo real sin bloquear el menú de la consola (se usan `TimerTask`).
- Modularidad: El código está organizado en 22 clases, cada una con una responsabilidad bien definida.
- Portabilidad: Compatible con cualquier sistema que soporte Java 8 y Oracle.
- Mantenibilidad: Las constantes del juego están centralizadas en la interfaz `Variables`.

2.3.Estructura del proyecto

Versión consola (Main.java)

- Menú principal con 8 opciones.
- Bucle infinito hasta que el usuario elige salir.
- Los temporizadores (ResourceGenerator, EnemyArmyCreator) se ejecutan en segundo plano.

Interfaz gráfica (JavaFX)

- Ventana principal con un TabPane de cuatro pestañas:
- Información: barra de recursos y dos columnas (estadísticas y ejército).
- Recursos y Edificios: construcción de edificios y mejora de tecnologías.
- Ejército: creación de unidades (ataque, defensa, especiales).
- Reportes: historial de batallas con marcador de victorias/derrotas y área de texto con el último informe.

Flujo del juego

- Inicio → Civilization() (recursos iniciales = 0).
- ResourceGenerator añade recursos cada minuto según edificios construidos.
- EnemyArmyCreator genera un ejército enemigo cada 3 minutos e inicia Battle.
- El jugador puede construir, mejorar, reclutar o consultar información en cualquier momento.
- Al final de una batalla, se actualiza el historial y se añaden residuos si gana la civilización.

2.4.Diseño de mecánicas

2.4.1.Recursos

- Comida – generación base: 8000/min + 4000 por granja.
- Madera – generación base: 5000/min + 2500 por carpintería.
- Hierro – generación base: 1500/min + 750 por herrería.
- Maná – generación base: 0 + 10 por torre mágica.

2.4.2.Edificios

Edificio	Coste			Bonificación
Granja	5000 🍌	10000 🍷	12000 🔨	+4000 comida/min
Carpintería	10000 🍌	10000 🍷	12000 🔨	+2500 madera/min
Herrería	5000 🍌	10000 🍷	12000 🔨	+750 hierro/min
Torre Mágica	5000 🍌	10000 🍷	12000 🔨	+10 maná/min, permite magos
Iglesia	5000 🍌	10000 🍷	12000 🔨	Permite sacerdotes

2.4.3.Tecnologías

- Ataque: aumenta el daño base de las nuevas unidades. Coste inicial: 2000 hierro; incremento: 60 hierro por nivel.
- Defensa: aumenta la armadura de las nuevas unidades. Mismo coste que ataque.

Unidades: Se dividen en tres categorías (ataque, defensa, especiales) que heredan de clases abstractas. Cada unidad conoce sus costes y sus probabilidades de generar residuos o atacar de nuevo. La experiencia y la santificación (otorgada por sacerdotes) aumentan el daño y la armadura.

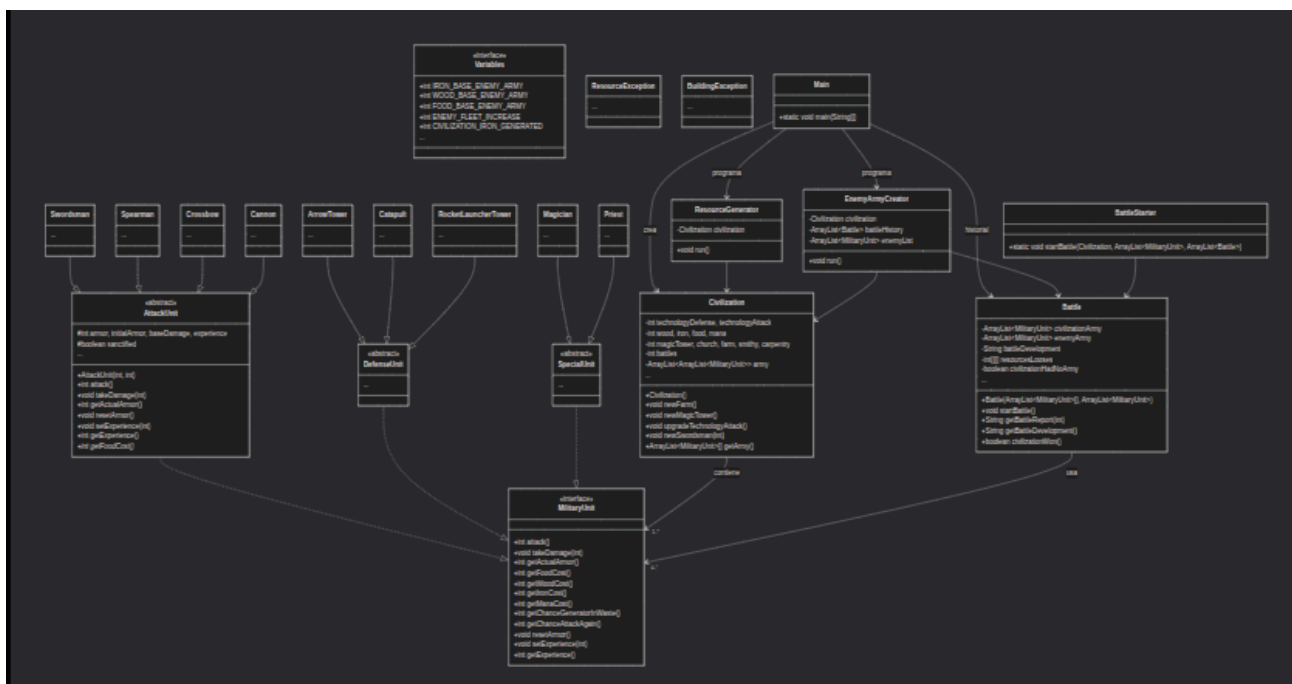
2.4.4.Batalla

- Se elige aleatoriamente qué ejército ataca primero.
- Se selecciona un grupo atacante según probabilidades definidas en Variables.

- Se selecciona un defensor mediante probabilidad proporcional al número de unidades de cada grupo.
- El atacante inflige su daño, el defensor reduce su armadura. Si armadura ≤ 0 , la unidad muere y puede generar residuos (madera y hierro según coste).
- Algunas unidades pueden atacar de nuevo (probabilidad).
- La batalla termina cuando un ejército se reduce al 10% de sus unidades iniciales.
- El ganador se decide por pérdidas ponderadas (1 hierro = 5 madera = 10 comida).

2.5. Diagrama de clases

El siguiente diagrama muestra las relaciones entre las interfaces, clases abstractas y clases concretas del proyecto. Se ha omitido la lista exhaustiva de atributos y métodos por claridad, representando solo los más significativos.

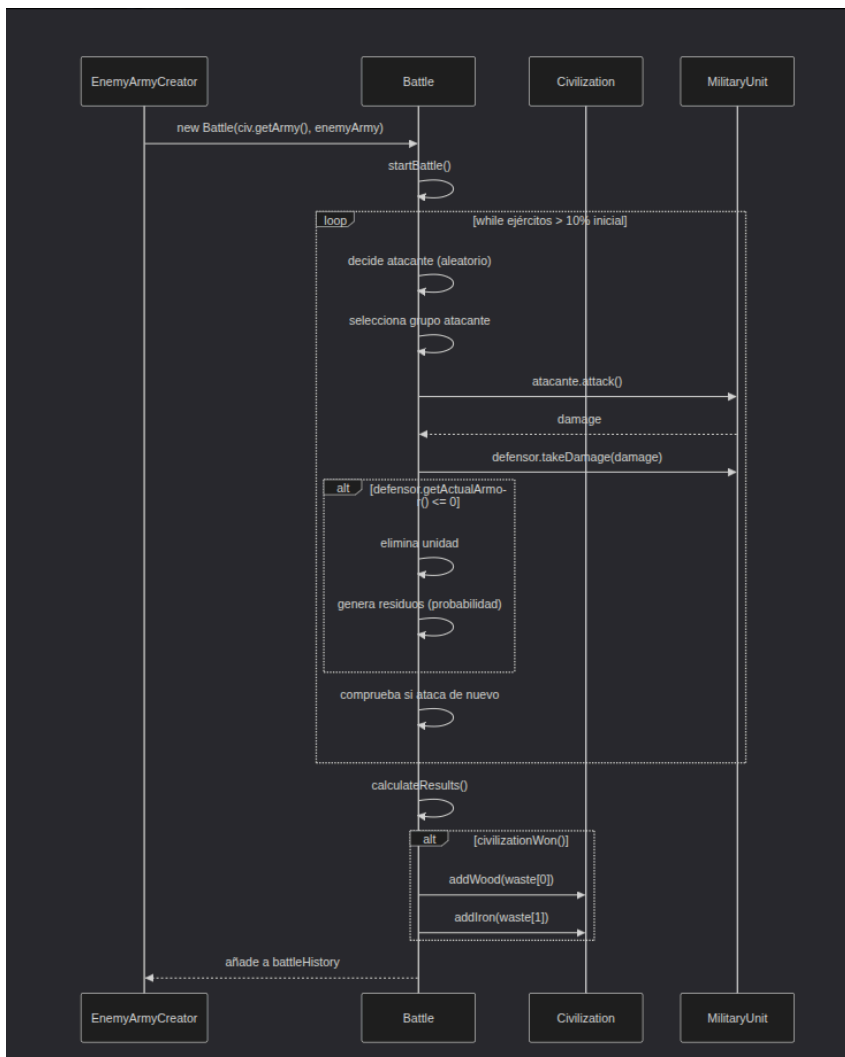


Explicación de la estructura:

- La interfaz MilitaryUnit define las operaciones comunes a todas las unidades.
- Las clases abstractas AttackUnit, DefenseUnit y SpecialUnit implementan parcialmente la interfaz y definen atributos comunes (armadura, daño, experiencia, santificado).
- Las 9 unidades concretas heredan de su clase abstracta correspondiente e implementan los métodos específicos de costes y probabilidades.
- Civilization agrupa recursos, edificios, tecnologías y el ejército (array de 9 ArrayList).
- Battle gestiona el combate entre dos ejércitos y genera reportes.
- ResourceGenerator y EnemyArmyCreator son TimerTask que se ejecutan periódicamente.

- Main es el punto de entrada de la versión consola. InterfazCivilization (en ui) es el punto de entrada de la interfaz gráfica.

2.5.1. Diagrama de secuencia simplificado de una batalla



3. Arquitectura y desarrollo

3.1. Estructura del código fuente

El código Java se organiza en el paquete civilizations. Las clases principales son:

Clase	Responsabilidad
Civilization	Gestiona toda la civilizacion,
Battle	Simula una batalla entre dos ejércitos.
EnemyArmyCreator	Crea el ejército enemigo cada 3 minutos.
ResourceGenerator	Añade recursos cada minuto.
Variables	Contiene todas las constantes del juego
MilitaryUnit (interfaz)	Define las operaciones de cualquier unidad.
AttackUnit, DefenseUnit, SpecialUnit	Clases abstractas base para las unidades.
9 clases de unidades concretas	Swordsman, Spearman, Crossbow, Cannon, ArrowTower, Catapult, RocketLauncherTower, Magician, Priest.
ResourceException, BuildingException	Excepciones personalizadas.
Main	Punto de entrada de la versión consola.
InterfazCivilization + ui/*	Interfaz gráfica JavaFX (4 paneles)

3.2.Componentes principales

Civilization: Contiene un array de 9 `ArrayList<MilitaryUnit>` que representa el ejército, uno por cada tipo de unidad. Los métodos `newSwordsman`, `newFarm`, etc., lanzan excepciones si no hay suficientes recursos o edificios requeridos. La generación de recursos se realiza mediante un `TimerTask` independiente.

Battle: En su constructor recibe el ejército de la civilización (como array de listas) y una lista plana del ejército enemigo. Internamente convierte todo a listas planas para el combate. Lleva un registro de las unidades iniciales y actuales para calcular pérdidas. Al final, decide el ganador por pérdidas ponderadas y genera un reporte textual.

EnemyArmyCreator: Se ejecuta cada 3 minutos. Calcula los recursos base del enemigo multiplicados por un factor ($2.5 + 0.4 * \text{número de batallas}$) para que sea más difícil con el tiempo. Crea unidades de ataque de forma aleatoria hasta agotar recursos, y luego añade algunas defensas (torres) proporcionalmente al tamaño del ejército. Si el ejército queda vacío, añade 20 espadachines de emergencia.

ResourceGenerator: Cada minuto añade recursos según la fórmula: $\text{base} + \text{edificios} * \text{bonus}$.

TimerTasks: Se programan en `Main` (consola) y en `MainWindow` (GUI) con `Timer.scheduleAtFixedRate`. Se ejecutan en segundo plano y actualizan la interfaz mediante `Platform.runLater`.

3.3. Diagrama de archivos

3.4 Flujo de datos

Inicio:

- Consola: `Main` crea `Civilization` y programa los `TimerTask`.
- GUI: `InterfazCivilization` lanza `MainWindow`, que hace lo mismo.

Generación de recursos (cada 1 minuto):

- `ResourceGenerator.run()` → `civilization.addFood(...)` etc.

Creación de ejército enemigo (cada 3 minutos):

- `EnemyArmyCreator.run()` → calcula recursos base con factor de dificultad → crea unidades aleatorias → añade defensas → crea `Battle` con `civilization.getArmy()` y el nuevo ejército → llama a `battle.startBattle()`.

Batalla:

- `Battle.startBattle()` → simula turnos → al final calcula resultado → si gana la civilización, añade residuos → añade batalla al historial.

Interacción del jugador:

- Consola: menú numérico.
- GUI: botones en pestañas que llaman a métodos de `Civilization` y actualizan la interfaz mediante `updateUI()`.

Web (datos estáticos de ejemplo):

- El servidor Express sirve páginas HBS con datos ficticios (pendiente de integrar con base de datos real).

4. Test y depuración

4.1. Pruebas realizadas

- Compilación: Todas las clases se compilan sin errores con JDK 17 y JavaFX 23.

- Ejecución de la versión consola: Se probaron todas las opciones del menú, incluyendo construir edificios sin recursos, crear unidades con requisitos incumplidos, y ver el historial de batallas.
- Ejecución de la interfaz gráfica: Se verificó que las pestañas se muestran correctamente, los botones ejecutan las acciones esperadas y los recursos se actualizan.
- Pruebas de batalla:
 - Se simularon múltiples batallas para comprobar que el enemigo crece en dificultad.
 - Se forzó la condición de no tener ejército para verificar que la civilización pierde.
- Pruebas de excepciones:
 - Crear una unidad sin recursos suficientes → ResourceException.
 - Crear un mago sin torre mágica → BuildingException.
 - Construir un edificio sin recursos → ResourceException.
- Pruebas de la web: Se comprobó el renderizado de las páginas y la navegación entre secciones.

4.2.Errores encontrados y soluciones

Error	Causa	Solución
-------	-------	----------

ClassNotFoundException al ejecutar JavaFX	La ruta de <code>--module-path</code> no apuntaba a la carpeta lib de JavaFX.	Se usó <code>/usr/share/openjfx/lib</code> (Linux) o se descargó el SDK de JavaFX.
IllegalArgumentException: duplicate children en GridPane (GUI)	Se añadía el mismo botón dos veces al usar <code>addRow</code> incorrectamente.	Se cambió a <code>add(boton, col, fila)</code> con coordenadas explícitas.
La civilización siempre ganaba, incluso sin ejército	La comparación de pérdidas ponderadas ($0 \leq 0$) declaraba victoria.	Se añadió el flag <code>civilizationHadNoArmy</code> en <code>Battle</code> y se modificó <code>civilizationWon()</code> para devolver <code>false</code> en ese caso.
El enemigo era demasiado débil en batallas avanzadas	El factor de crecimiento era lineal y bajo (6% por batalla).	Se cambió a un factor multiplicador $2.5 + 0.4 * battles$ y se añadieron defensas enemigas.
La interfaz gráfica no reflejaba cambios en recursos tras construir	<code>updateUI()</code> no se llamaba después de la acción.	Se añadió <code>updateUICallback.run()</code> en cada <code>setOnAction</code> .
Los botones de mejora de tecnología no mostraban coste dinámico	Se calculaba el coste en <code>updateUI()</code> pero no se actualizaba el texto del botón.	Se guardó la referencia al <code>Label</code> del coste y se actualiza cada segundo.

4.3.Mejoras aplicadas durante el desarrollo

- Refactorización del ejército: De un `ArrayList<MilitaryUnit>` plano a un array de 9 `ArrayList<MilitaryUnit>`, uno por tipo, para facilitar el cálculo de probabilidades de defensor.

- Ajuste del umbral de batalla: Del 20% al 10% para hacer los combates más largos y dar oportunidad al enemigo.
- Incremento de dificultad del enemigo: Factor multiplicador en recursos y adición de defensas (torres) para equilibrar el juego.
- Mejora de la interfaz gráfica: Se añadió una pestaña de "Información" con barra de recursos y dos columnas (estadísticas y ejército), y se estilizó con colores oscuros y bordes azules.
- Manejo de excepciones unificado: Se crearon las interfaces funcionales `ActionWithException` en los paneles para capturar `ResourceException` y `BuildingException`.

5. Recursos gráficos y de audio

5.1 Fuentes utilizadas

- Emojis: Se emplearon emojis universales (🍷, 🍺, 🛠️, 💎) que se muestran correctamente en la mayoría de sistemas. Para la interfaz gráfica se usaron también emojis para representar recursos y acciones.
- Iconos web: Se utilizaron imágenes propias (logo, favicon, fotos de perfil) en formato PNG y JPG, ubicadas en `M04_Web/public/img/`.
- Fuentes tipográficas: En la web se usa `system-ui` y `Segoe UI`. En la GUI se usa la fuente por defecto del sistema (Arial) con estilos CSS.

6. Manual de usuario

6.1. Cómo ejecutar el juego

6.1.1. Versión por consola (sin interfaz gráfica)

- Asegúrate de tener Java JDK 17+ instalado.
- Descarga o clona el repositorio desde GitHub.
- Abre una terminal y navega a la carpeta `M03_Programacion/`.

- Compila todas las clases:
 - `javac civilizations/*.java`
- Ejecuta la clase Main:
 - `java civilizations.Main`

6.1.2. Interfaz gráfica (JavaFX)

- Instala JavaFX (versión 23). En Linux: `sudo apt install openjfx`. En Windows: descarga el SDK.
- Navega a `M03_Programacion/`.
- Compila incluyendo el módulo de JavaFX:
 - `javac --module-path /ruta/a/javafx/lib --add-modules javafx.controls,javafx.fxml -d . civilizations/*.java civilizations/ui/*.java`
- Ejecuta la clase `InterfazCivilization`:
 - `java --module-path /ruta/a/javafx/lib --add-modules javafx.controls,javafx.fxml -cp . civilizations.InterfazCivilization`

(Reemplaza `/ruta/a/javafx/lib` por la ubicación real de JavaFX.)

6.2 Controles y objetivo del juego

Objetivo: Sobrevivir el mayor número de batallas posible, optimizando la producción de recursos y el ejército.

6.2.1. Versión consola

- Usa el teclado numérico para seleccionar opciones del menú.

- Introduce cantidades cuando se te solicite.

6.2.2. Interfaz gráfica

- Navega por las pestañas usando el ratón.
- Haz clic en los botones para construir edificios, mejorar tecnologías o crear unidades.
- En la pestaña "Ejército", cada botón de unidad abre un diálogo para introducir la cantidad.

6.3 Explicación de las opciones del menú

6.3.1. Menú de consola

1. Ver estado de la civilización

Muestra recursos, edificios, tecnologías, ejército (número de unidades por tipo) y batallas libradas.

2. Construir edificios

Permite elegir entre granja, carpintería, herrería, torre mágica o iglesia. Verifica recursos suficientes.

3. Mejorar tecnologías

Mejora ataque o defensa. El coste aumenta con el nivel.

4. Crear unidades

Selecciona tipo de unidad y cantidad. Si no hay recursos suficientes, se crean las máximas posibles y se muestra un mensaje de advertencia.

5. Ver amenaza enemiga

Muestra la composición del próximo ejército enemigo (si ya ha sido generado).

6. Ver historial de batallas

Lista todas las batallas y permite ver el desarrollo paso a paso de cada una.

7. Ayuda

Muestra consejos y explicación de mecánicas.

8. Salir

Termina el juego.

6.3.2. Interfaz gráfica (pestañas)

- Información: Barra de recursos (comida, madera, hierro, maná) en la parte superior. Debajo, dos columnas: izquierda con estadísticas (tecnologías y edificios) y derecha con el ejército desglosado.
- Recursos y Edificios: Panel central con botones para construir cada edificio y para mejorar tecnologías. Los recursos se muestran en la barra superior.
- Ejército: Barra de recursos (igual). Cuadrícula de 3x3 con iconos y nombres de unidades, pero sin contadores (se eliminaron por petición del usuario). Abajo, botones para crear unidades, organizados en "Unidades de ataque", "Defensas" y "Unidades especiales".
- Reportes: Muestra el marcador de victorias/derrotas (estilo fútbol) y el reporte completo de la última batalla (estadísticas y desarrollo) en un área de texto negra

7. Reflexión y mejoras

7.1 Dificultades encontradas y cómo se superaron

- Sincronización de temporizadores con la interfaz gráfica: Los `TimerTask` se ejecutan en hilos separados, y actualizar la UI desde ellos causaba excepciones. Se solucionó usando `Platform.runLater()` para envolver las actualizaciones.
- Complejidad de la batalla: Implementar la selección aleatoria de atacante, grupo atacante (según probabilidades fijas) y defensor (según cantidad de unidades) requirió varios métodos auxiliares. Se modeló siguiendo el algoritmo explicado en el PDF original.
- Santificación de unidades: La mecánica de santificación (mejora de stats por sacerdotes) no se completó por falta de tiempo, pero se dejaron los atributos `sanctified` en `AttackUnit` y `DefenseUnit` preparados para una futura implementación.
- Equilibrio de juego: Al principio la civilización ganaba siempre. Se solucionó aumentando la dificultad del enemigo (factor de recursos y defensas) y reduciendo el umbral de fin de batalla al 10%.
- Manejo de excepciones en lambdas de JavaFX: Los métodos `newSwordsman`, etc., lanzan excepciones verificadas, pero `setOnAction` no las permite. Se creó una interfaz funcional `ActionWithException` y se capturaron dentro del lambda.

7.2 Mejoras futuras

- Santificación completa: Implementar que los sacerdotes puedan santificar un número limitado de unidades, mejorando sus estadísticas (armadura y ataque).
- Coste para resetear armaduras: Actualmente `resetCivilizationArmor()` es gratuito; se podría añadir un coste en recursos.
- Múltiples civilizaciones enemigas: En lugar de un único enemigo, diferentes facciones que ataquen con intervalos aleatorios y estilos de ejército distintos.
- Límite de nivel de tecnología: Para evitar desequilibrios, se podría establecer un nivel máximo (por ejemplo, 10).
- Mantenimiento de unidades: Algunas unidades podrían requerir comida periódicamente; si no la hay, mueren.
- Persistencia completa con base de datos: Conectar la aplicación web con la base de datos real (Oracle) para mostrar datos actualizados de las partidas.

- Mejora de la interfaz gráfica: Añadir un mapa interactivo (tipo RTS) con desplazamiento y selección de unidades (se inició en WorldMapView pero no se completó).
- Efectos de sonido y música: Incorporar recursos de audio para inmersión.